

TRI DES PROCESSUS PRETS SELON LE TEMPS RESTANT

```
PCB* op_sort_ready_by_rt(PROCESS_MANAGER* self) { //Fonction qui retourne un processus (le premier trié) => trier les processus prêts selon leur temps d'exécution restant.
```

```
    PCB* ready_queue_head = self->ready_queue_head; //Récupere la tête de de la file des processus prêts.
```

```
    if (ready_queue_head == NULL) { // si la file prete (ready queue )est vide donc il n y a aucun processus a trier.
```

```
        return NULL;
```

```
    }
```

```
    // check if ready is circular to make the break
```

```
    PCB* end = ready_queue_head; // end va nous servir a parcourir la liste jusqu'au dernier élément
```

```
    bool circular = false; // par défaut on suppose que la liste n'est pas circulaire.
```

```
    // check circular and find end
```

```
    while (end->pid_sibling_next != NULL && end->pid_sibling_next != ready_queue_head)
```

```
{ //tant qu'on est pas encore à la fin de la liste normale et qu'on est pas revenu au début, la liste n'est pas circulaire.
```

```
    end = end->pid_sibling_next;
```

```
}
```

```
if (end->pid_sibling_next == ready_queue_head) { // it's circular so break
```

```
    circular = true;
```

```
    end->pid_sibling_next = NULL;
```

```
    printf("DEBUG: circular list, breaking circle\n");
```

```
}
```

```
    // sorting
```

```
    PCB* sorted_head = NULL; //au debut on n'a encore rien trié donc la tete de la liste triée est vide.
```

```
    PCB* current = ready_queue_head; // parcourir l'ancienne liste(non triée).
```

```
while (current != NULL) {  
    PCB* next = current->pid_sibling_next; // next memorise le processus suivant avant de casser les  
    liens.
```

```
    // insert to sorted list
```

```
    if (sorted_head == NULL || current->remaining_time < sorted_head->remaining_time) { //si la  
    liste triée est vide et le processus courant a un temps restant plus petit.
```

```
        current->pid_sibling_next = sorted_head;
```

```
        sorted_head = current; //current devient la nouvelle tete de la liste triée
```

resultat visuel :

avant : sorted_head → P2 → P3

apres: sorted_head → P1 → P2 → P3

```
    } else {
```

```
        PCB* search = sorted_head; //search va server a trouver la position d'insertion.
```

```
        while (search->pid_sibling_next != NULL && search->pid_sibling_next->prioritie < current-  
>prioritie) { //tant qy'on est pas a la fin et que le prochain élément doit rester avant current
```

```
            search = search->pid_sibling_next; //on avance
```

```
        }
```

```
        current->pid_sibling_next = search->pid_sibling_next;
```

```
        search->pid_sibling_next = current;
```

```
    }
```

```
    current = next;
```

```
}
```

```
if (circular && sorted_head != NULL) { //si la liste etait circulaire au départ et qu'elle n'est pas vide
```

```
    PCB* new_tail = sorted_head; // remake the list circular
```

```
    while (new_tail->pid_sibling_next != NULL) { // find the last element
```

```
        new_tail = new_tail->pid_sibling_next;
```

```
    }
```

```
new_tail->pid_sibling_next = sorted_head; // then push it
}

return sorted_head;
}
```